THE DEVELOPER'S HANDBOOK | EDITION 1.0

Docker

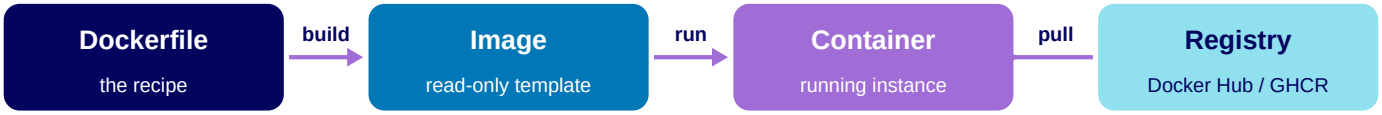
Developer Guide

Build, ship and run applications anywhere.
Core concepts, commands, Dockerfiles, Compose and best practices.

By Dinesh

01 What is Docker?

Docker packages your application and everything it needs (runtime, libraries, config) into a **container**: a lightweight, isolated process that runs identically on your laptop, CI server and production. Unlike virtual machines, containers share the host kernel, so they start in **seconds**, weigh **megabytes** and let you run dozens on one machine. No more *"works on my machine"*.



	Container	Virtual Machine
`Startup`	Seconds	Minutes
`Size`	Megabytes	Gigabytes
`Isolation`	Process level (shared kernel)	Full guest OS

02 Core Concepts

Term	What it is	Think of it as
`Image`	Read-only template with your app, runtime and dependencies, built in layers.	A class
`Container`	A running (or stopped) instance of an image with its own filesystem and network.	An object
`Dockerfile`	Text file of instructions that tells Docker how to build an image.	Source code
`Registry`	Server that stores and distributes images (Docker Hub, GHCR, AWS ECR).	npm for images
`Volume`	Persistent storage that outlives the container.	An external disk
`Network`	Virtual network that lets containers find and talk to each other.	A private LAN

03 Install and First Run

Install **Docker Desktop** (Windows / macOS) or **Docker Engine** (Linux), then verify and run your first container:

terminal

```
docker --version
docker run hello-world                    # sanity check
docker run -d -p 8080:80 --name web nginx # web server in 1 line
# open http://localhost:8080 in your browser
docker stop web && docker rm web          # clean up
```

(c) 2026 Dinesh. Licensed for personal and professional use. Redistribution prohibited.

04 Essential Commands

Command	What it does
`docker build -t app:1.0 .`	Build an image from the Dockerfile in the current folder
`docker images | pull | push <image>`	List local images / download from / upload to a registry
`docker run -d -p 3000:3000 app:1.0`	Start a container in the background, map host:container port
`docker ps -a`	List all containers, including stopped ones
`docker logs -f <name>`	Stream a container's logs live
`docker exec -it <name> sh`	Open an interactive shell inside a running container
`docker stop / start / restart <name>`	Control the container lifecycle
`docker rm -f <name>`	Force-remove a container
`docker system prune -a`	Delete all unused images, containers and networks (careful!)

Handy `docker run` flags

Flag	Purpose	Flag	Purpose
`-d`	Run detached (background)	`-e KEY=val`	Set environment variable
`-p 8080:80`	Publish host:container port	`-v data:/path`	Mount volume or folder
`--name x`	Give the container a name	`--network n`	Join a network
`--rm`	Auto-delete when it stops	`-it`	Interactive terminal

05 Writing a Dockerfile

Instruction	Purpose
`FROM`	Base image to start from, e.g. node:20-alpine
`WORKDIR`	Set the working directory for the following steps
`COPY`	Copy files from your machine into the image
`RUN`	Execute a command at build time (install packages, compile)
`ENV / ARG`	Runtime environment variable / build-time variable
`EXPOSE`	Document the port the app listens on
`CMD / ENTRYPOINT`	Default command at start / fixed executable that CMD can feed arguments to

Dockerfile (Node.js example)

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./              # dependencies first = cached layer
RUN npm ci --omit=dev
COPY . .
EXPOSE 3000
USER node                          # never run as root
CMD ["node", "server.js"]
```

PRO TIP: LAYER CACHING

Docker caches each instruction as a layer. Put things that change rarely (dependency files) **above** things that change often (your source code) so rebuilds take seconds, not minutes.

(c) 2026 Dinesh. Licensed for personal and professional use. Redistribution prohibited.

06 Volumes and Persistent Data

Container filesystems are **ephemeral**: delete the container and the data is gone. Use volumes for anything you need to keep.

Type	Use it for	Example
Named volume	Databases and production data, managed by Docker	`-v pgdata:/var/lib/postgresql/data`
Bind mount	Live-reload in development, share a host folder	`-v $(pwd):/app`
tmpfs	Sensitive or temporary data kept only in memory	`--tmpfs /tmp`

```
docker volume create pgdata
docker run -d --name db -v pgdata:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=example postgres:16
docker volume ls                        # list volumes
```

07 Networking

Containers on a **user-defined bridge network** reach each other by **container name**. No IP addresses needed.

```
docker network create appnet
docker run -d --network appnet --name db -e POSTGRES_PASSWORD=example postgres:16
docker run -d --network appnet -p 3000:3000 -e DB_HOST=db app:1.0
```

Driver	Behaviour
`bridge`	Default. Isolated private network, ports published with -p
`host`	Shares the host network stack, no port mapping (Linux)
`none`	No networking at all, fully isolated

08 Docker Compose: Multi-Container Apps

Describe your whole stack in one **compose.yaml** and control it with a single command.

compose.yaml

```
services:
  api:
    build: .
    ports: ["3000:3000"]
    environment:
      DB_HOST: db
    depends_on: [db]
  db:
    image: postgres:16
    environment:
      POSTGRES_PASSWORD: example
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Everyday Compose commands

Command	Action
`docker compose up -d --build`	Build and start all services
`docker compose ps`	Show service status
`docker compose logs -f api`	Follow one service's logs
`docker compose exec api sh`	Shell into a service
`docker compose down -v`	Stop and remove (incl. volumes)

(c) 2026 Dinesh. Licensed for personal and professional use. Redistribution prohibited.

09 Multi-Stage Builds: Small, Secure Images

Build in one stage, ship only the output in another. Your production image carries **no compilers, no source, no dev tools**, often 10x smaller.

Dockerfile (multi-stage)

```
FROM node:20-alpine AS build              # stage 1: build
WORKDIR /app
COPY . .
RUN npm ci && npm run build

FROM nginx:alpine                          # stage 2: tiny runtime
COPY --from=build /app/dist /usr/share/nginx/html
EXPOSE 80
```

10 Production Best Practices

- **Pin image versions**
  Use node:20.11-alpine, never :latest.
- **Use small base images**
  alpine or -slim cut size and attack surface.
- **Add a .dockerignore**
  Exclude node_modules, .git and .env files.
- **Run as non-root**
  Add USER to limit damage if compromised.
- **One process per container**
  Easier to scale, restart and monitor.
- **Never bake in secrets**
  Inject at runtime via env vars or secret stores.
- **Add a HEALTHCHECK**
  Let orchestrators detect and restart broken apps.
- **Scan your images**
  Run docker scout cves <image> before shipping.

11 Troubleshooting Cheat Sheet

Symptom	Likely cause	Fix
Container exits immediately	Main process crashed or finished	docker logs <name>, then run with -it
port is already allocated	Host port in use	Change the host side: -p 8081:80
Cannot connect to the Docker daemon	Docker not running / no permission	Start Docker; sudo usermod -aG docker $USER
Code changes not showing	Stale cache or old image	docker build --no-cache, then recreate
No space left on device	Unused images and volumes pile up	docker system df, then docker system prune
Containers can't reach each other	Different networks	Same user-defined network, use the service name

12 Power One-Liners

terminal

```
docker stats                                  # live CPU / memory per container
docker inspect -f '{{.NetworkSettings.IPAddress}}' <name>
docker cp <name>:/app/app.log .               # copy a file out of a container
docker image history app:1.0                  # see layers and their sizes
docker tag app:1.0 user/app:1.0 && docker push user/app:1.0
```

Ship it. Containerize your next project today.

Next steps: Kubernetes, CI/CD pipelines, image registries and cloud deployment.

Docker Developer Guide | Written by Dinesh

(c) 2026 Dinesh. Licensed for personal and professional use. Redistribution prohibited.